CENTRO UNIVERSITÁRIO CATÓLICA DE SANTA CATARINA
Engenharia de Software — Programação Web

LocaFácil

Sistema de Gestão de Aluguel de Veículos

Projeto Final — Etapa 1: Documento de Concepção, Requisitos e Modelagem de Dados

Identificação da equipe

Integrante	Função
BRUNO ROVANI MARCELINO	Autenticação e segurança
BRUNO SCHIMIGUEL	Cadastro e gerenciamento da frota
RENAN GABRIEL PIECHONTCOSKI	Clientes e reservas
GABRIEL HOEFT TISSI	Gestão de locações e pagamentos
Líder do grupo	**BRUNO ROVANI MARCELINO**

Repositório Git: https://github.com/brunow27/projeto_web

Jaraguá do Sul — outubro de 2026

Sumário

1. Visão Geral do Projeto 3
1.1 Nome do Produto 3
1.2 Declaração do Problema e Justificativa 3
1.3 Público-Alvo e Objetivos do Sistema 3
1.4 Tecnologias 3
2. Especificação de Requisitos 4
2.1 Requisitos Funcionais 4
2.2 Requisitos Não Funcionais 4
2.3 Regras de Negócio 5
3. Arquitetura e Modelagem de Dados 6
3.1 Diagrama de Classes UML 6
3.2 Mapeamento de Entidades JPA e Dicionário de Dados 7
3.3 Repositórios Spring Data JPA 9
3.4 Matriz de Rastreabilidade 10
4. Repositório e Execução 10

1. Visão Geral do Projeto

1.1 Nome do Produto

LocaFácil — Sistema de Gestão de Aluguel de Veículos. Trata-se de uma API REST desenvolvida em Java com Spring Boot para controlar frota, clientes, reservas, locações e pagamentos de uma locadora de carros.

1.2 Declaração do Problema e Justificativa

Locadoras de pequeno e médio porte ainda controlam reservas e frota em planilhas e mensagens. Esse controle manual gera conflitos de agenda (o mesmo carro reservado duas vezes para o mesmo período), cobrança incorreta de diárias e multas, e ausência de histórico confiável por cliente e por veículo.

O LocaFácil centraliza esses dados em uma base única e aplica as regras de negócio de forma automática: bloqueia reservas sobrepostas, valida a CNH do condutor, calcula o valor da locação (diárias, opcionais e multa por atraso) e registra os pagamentos, informando o saldo devedor de cada locação.

1.3 Público-Alvo e Objetivos do Sistema

Público-alvo: atendentes e gerentes de locadoras (uso interno), administradores do sistema e clientes pessoa física que reservam veículos. O sistema trabalha com três perfis de acesso: `ADMIN`, `ATENDENTE` e `CLIENTE`.

Proposta de valor: reduzir erros operacionais e retrabalho, garantindo que nenhum veículo seja reservado em duplicidade e que todo valor cobrado siga as regras da categoria do veículo.

Objetivos:

- Manter o cadastro de clientes, habilitações, veículos, categorias e filiais.
- Garantir a disponibilidade correta dos veículos por período (reservas, locações ativas e manutenções).
- Calcular o valor total da locação (diárias, opcionais e multa por atraso) e controlar pagamentos.
- Expor uma API REST documentada e protegida por autenticação, com autorização por perfil.
- Persistir os dados com JPA/Hibernate, com 11 entidades e relacionamentos 1:1, 1:N e N:M.

1.4 Tecnologias

Camada	Tecnologia
Linguagem	Java 17
Framework	Spring Boot 3.3 (Web, Data JPA, Validation; Security na Etapa 2)
Persistência	JPA / Hibernate, Spring Data JPA (`JpaRepository`)
Banco de dados	PostgreSQL (execução) · H2 em memória (desenvolvimento e testes)
Autenticação	JWT (stateless), senhas com BCrypt
Documentação da API	OpenAPI / Swagger (springdoc)
Testes	JUnit 5, Mockito, `@DataJpaTest`
Build / Versionamento	Maven · Git (GitHub)

Tabela 1 — Tecnologias adotadas

2. Especificação de Requisitos

2.1 Requisitos Funcionais

Os requisitos funcionais descrevem o que o sistema deve fazer. A coluna “Perfil” indica quais perfis de usuário podem executar cada funcionalidade.

Código	Requisito	Descrição	Perfil
RF01	Autenticação de usuários	Login com e-mail e senha, retorna token JWT com expiração. Todas as rotas, exceto login e cadastro de cliente, exigem token.	Todos
RF02	Gerenciamento de usuários	CRUD de usuários, desativação e troca de perfil. E-mail único e senha com hash.	ADMIN
RF03	Cadastro de clientes e habilitação	CRUD de clientes (nome, CPF único, telefone, endereço), cada um com uma habilitação (CNH, categoria, validade). Autocadastro permitido.	ADMIN, ATENDENTE, CLIENTE
RF04	Gerenciamento de categorias	CRUD de categorias (Econômico, SUV, Executivo) com diária, multa por dia de atraso e caução.	ADMIN
RF05	Gerenciamento de filiais	CRUD de filiais. Não pode remover filial com veículos ou locações associados.	ADMIN
RF06	Gerenciamento de veículos	CRUD de veículos (placa única, marca, modelo, ano, quilometragem, status) ligados a uma categoria e a uma filial.	ADMIN, ATENDENTE
RF07	Consulta de disponibilidade	Lista paginada de veículos disponíveis em um período, com filtros por filial, categoria e preço.	Todos
RF08	Gerenciamento de opcionais	CRUD de opcionais (GPS, cadeirinha, seguro adicional) com valor por dia.	ADMIN
RF09	Criação de reserva	Cria locação `RESERVADA` validando disponibilidade e CNH, com opcionais e valor previsto.	ADMIN, ATENDENTE, CLIENTE
RF10	Retirada do veículo	Registra km inicial e data/hora real; locação vira `ATIVA` e veículo `ALUGADO`.	ADMIN, ATENDENTE
RF11	Devolução do veículo	Registra km final e data/hora real, calcula multa por atraso e valor final; locação vira `FINALIZADA`.	ADMIN, ATENDENTE
RF12	Cancelamento de reserva	Cancela locação ainda `RESERVADA`, com motivo, liberando o período.	ADMIN, ATENDENTE, CLIENTE
RF13	Registro de pagamentos	Registra sinal, saldo ou multa (PIX, cartão, dinheiro) e informa o saldo devedor.	ADMIN, ATENDENTE
RF14	Gerenciamento de manutenções	Registra manutenções com período, tipo e custo; veículo fica indisponível no período.	ADMIN, ATENDENTE
RF15	Histórico e consultas	Lista locações por cliente, veículo, filial, status e período; histórico de veículo. Cliente vê só as próprias.	ADMIN, ATENDENTE, CLIENTE

Tabela 2 — Requisitos funcionais

2.2 Requisitos Não Funcionais

Os requisitos não funcionais definem restrições técnicas de tecnologia, arquitetura, banco de dados, segurança, padrões REST, desempenho e qualidade.

Código	Categoria	Requisito
RNF01	Tecnologia	Java 17, Spring Boot 3.x, Maven, código versionado em Git.
RNF02	Arquitetura	Camadas `controller`, `service`, `repository`, `entity`, `dto`. Entidades JPA não são expostas na API (uso de DTOs).
RNF03	Banco de dados	JPA/Hibernate; PostgreSQL em execução e H2 nos testes; chaves primárias/estrangeiras e unicidade (CPF, placa, e-mail).

Código	Categoria	Requisito
RNF04	Banco de dados	Operações multi-tabela em uma única transação (`@Transactional`); `BigDecimal` para dinheiro; `LocalDate`/`LocalDateTime` para datas.
RNF05	Segurança	JWT stateless com autorização por perfil; 401 sem token, 403 sem permissão.
RNF06	Segurança	Senhas com BCrypt; segredos (chave JWT, credenciais do banco) via variáveis de ambiente.
RNF07	Validação	Bean Validation com resposta 400 por campo; `@RestControllerAdvice` global com corpo de erro padronizado.
RNF08	REST	Recursos no plural (`/clientes`, `/veiculos`, `/locacoes`), verbos HTTP corretos, JSON e status 200/201/204/400/401/403/404/409.
RNF09	Desempenho	Listagens paginadas (`Pageable`); `JOIN FETCH`/`@EntityGraph` para evitar N+1; leitura simples em até 500 ms com 10 mil veículos.
RNF10	Documentação	Swagger UI em `/swagger-ui.html` e este README.
RNF11	Qualidade	Testes unitários (JUnit 5/Mockito) das regras críticas e `@DataJpaTest` nas consultas.
RNF12	Integridade	Exclusão lógica (`ativo`) para clientes, usuários e veículos com histórico.

Tabela 3 — Requisitos não funcionais

2.3 Regras de Negócio

Código	Regra
RN01	Um veículo não pode ter duas locações (`RESERVADA` ou `ATIVA`) nem manutenção em períodos que se sobrepõem.
RN02	Não é possível reservar com CNH vencida na data de devolução prevista.
RN03	Valor previsto = diárias da categoria × dias + soma dos opcionais (valor/dia × quantidade × dias).
RN04	Na devolução, atraso gera multa = dias de atraso × `valorMultaDia` da categoria.
RN05	Transições de status válidas: `RESERVADA → ATIVA → FINALIZADA`; `RESERVADA → CANCELADA`.
RN06	Soma dos pagamentos não pode ultrapassar o valor da locação; o saldo devedor é informado na consulta.
RN07	Clientes só enxergam as próprias locações.

Tabela 4 — Regras de negócio

3. Arquitetura e Modelagem de Dados

A aplicação segue arquitetura em camadas (`controller` → `service` → `repository` → `entity`), com DTOs na fronteira da API. Nesta Etapa 1 foram implementadas as camadas de domínio (`entity`) e persistência (`repository`), no pacote `br.edu.catolicasc.locafacil`.

3.1 Diagrama de Classes UML

O diagrama abaixo apresenta as 11 classes de domínio com atributos tipados, métodos principais, relacionamentos e cardinalidades. Os enums `Perfil`, `StatusVeiculo`, `StatusLocacao`, `FormaPagamento` e `TipoPagamento` tipam os atributos de estado.

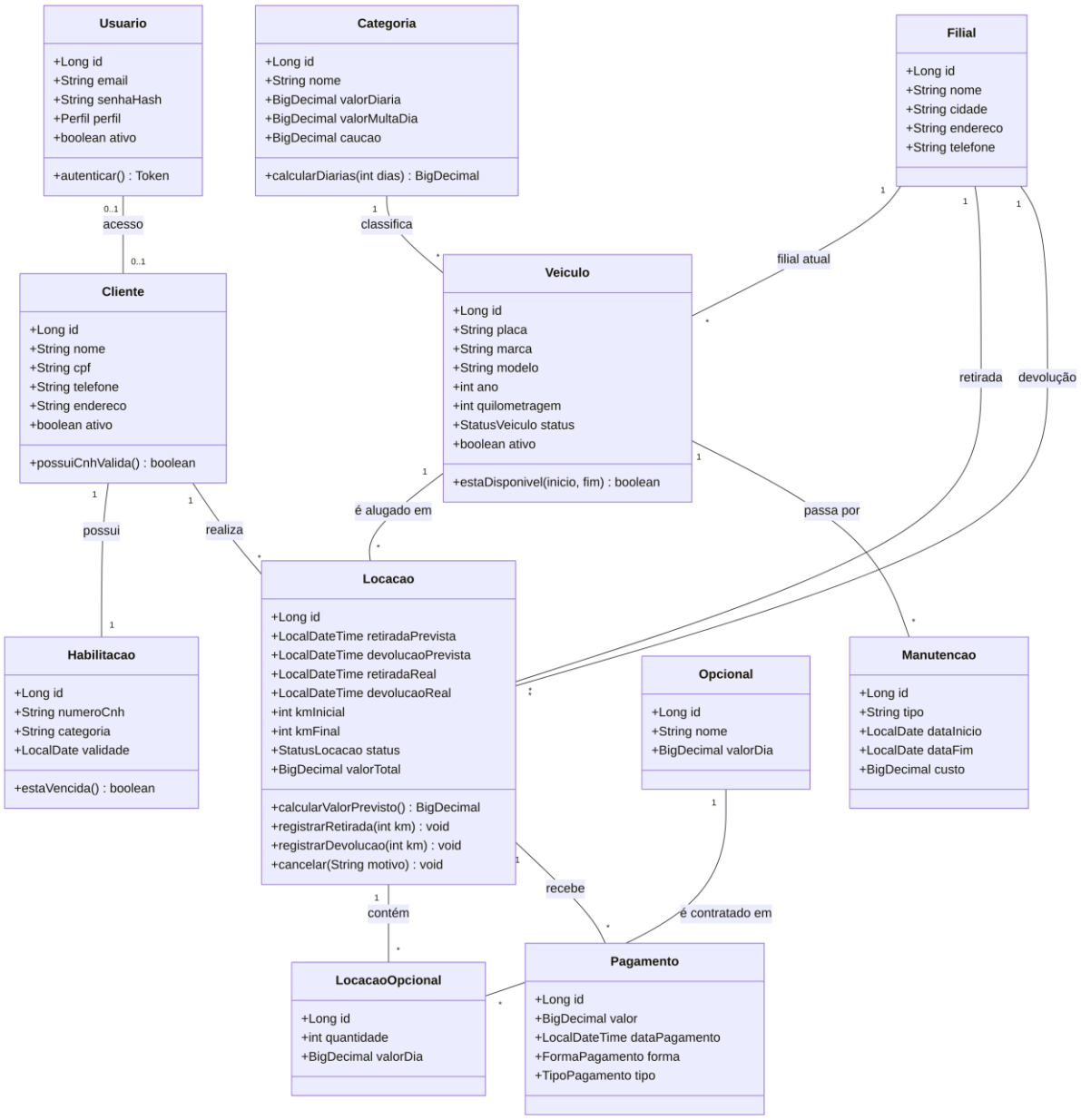


Figura 1 — Diagrama de classes UML do LocaFácil

Relacionamentos, cardinalidades e mapeamento JPA:

Relacionamento	Tipo	Anotações JPA
Cliente — Habilitacao	1:1	@OneToOne(cascade = ALL, orphanRemoval = true) `em` Cliente

Relacionamento	Tipo	Anotações JPA
Usuario — Cliente	1:1 (opcional)	@OneToOne em Cliente (lado dono, usuario_id)
Cliente — Locacao	1:N	@ManyToOne em Locacao / @OneToMany(mappedBy = "cliente") em Cliente
Veiculo — Locacao	1:N	@ManyToOne em Locacao
Categoria — Veiculo	1:N	@ManyToOne em Veiculo
Filial — Veiculo	1:N	@ManyToOne em Veiculo
Filial — Locacao (retirada e devolução)	1:N (duas associações)	dois @ManyToOne em Locacao: filialRetirada e filialDevolucao
Locacao — Opcional	N:M	tabela associativa LocacaoOpcional (com quantidade e valorDia), via dois @ManyToOne
Locacao — Pagamento	1:N	@ManyToOne em Pagamento
Veiculo — Manutencao	1:N	@ManyToOne em Manutencao

Tabela 5 — Relacionamentos e anotações JPA

A relação N:M entre `Locacao` e `Opcional` é modelada pela entidade associativa `LocacaoOpcional`, pois a associação possui atributos próprios (quantidade e valor cobrado por dia no momento da reserva), o que não é possível com um `@ManyToMany` simples.

3.2 Mapeamento de Entidades JPA e Dicionário de Dados

Entidades persistidas: foram implementadas 11 entidades anotadas com `@Entity` (o mínimo exigido é 5): `Usuario`, `Cliente`, `Habilitacao`, `Filial`, `Categoria`, `Veiculo`, `Locacao`, `Opcional`, `LocacaoOpcional`, `Pagamento` e `Manutencao`.

Convenções de mapeamento:

- Chave primária `Long id` com `@GeneratedValue(strategy = IDENTITY)` em todas as entidades.
- Valores monetários em `BigDecimal` com `precision = 10, scale = 2`; datas em `LocalDate`/`LocalDateTime`.
- Enums persistidos como texto com `@Enumerated(EnumType.STRING)`.
- Unicidade garantida por `@Column(unique = true)` (CPF, placa, e-mail, CNH) e `@UniqueConstraint` (par locação/opcional).
- Associações `@ManyToOne(fetch = LAZY)` para evitar carregamento desnecessário; `@EntityGraph` e `JOIN FETCH` nas consultas que precisam dos dados relacionados (evita N+1).
- Bean Validation (`@NotNull`, `@NotBlank`, `@Email`, `@Pattern`, `@DecimalMin`, `@Min`) nos atributos.
- Exclusão lógica pelo campo `ativo` em `Usuario`, `Cliente` e `Veiculo`.

Exemplo de mapeamento (trecho de `Locacao.java`):

```
@ManyToOne(fetch = FetchType.LAZY, optional = false)
@JoinColumn(name = "veiculo_id", nullable = false)
private Veiculo veiculo;

@OneToMany(mappedBy = "locacao", cascade = CascadeType.ALL, orphanRemoval = true)
private List<LocacaoOpcional> opcionais = new ArrayList<>();
```

Dicionário de dados:

Entidade (tabela)	Campo	Tipo	Restrições
Usuario (usuarios)	id	Long	PK, auto incremento
	email	String(120)	único, não nulo
	senhaHash	String(100)	não nulo (BCrypt)
	perfil	Enum Perfil	ADMIN, ATENDENTE, CLIENTE

Entidade (tabela)	Campo	Tipo	Restrições
	ativo	boolean	padrão `true`
Cliente (`clientes`)	id	Long	PK
	nome	String(120)	não nulo
	cpf	String(11)	único, não nulo
	telefone	String(20)	
	endereco	String(200)	
	ativo	boolean	padrão `true`
	habilitacao_id	Long	FK → `habilitacoes`, único
	usuario_id	Long	FK → `usuarios`, único, opcional
Habilitacao (`habilitacoes`)	id	Long	PK
	numeroCnh	String(20)	único, não nulo
	categoria	String(3)	não nulo (A, B, AB…)
	validade	LocalDate	não nulo
Categoria (`categorias`)	id	Long	PK
	nome	String(60)	único, não nulo
	valorDiaria	BigDecimal(10,2)	> 0
	valorMultaDia	BigDecimal(10,2)	≥ 0
	caucao	BigDecimal(10,2)	≥ 0
Filial (`filiais`)	id	Long	PK
	nome	String(80)	não nulo
	cidade	String(80)	não nulo
	endereco	String(200)	
	telefone	String(20)	
Veiculo (`veiculos`)	id	Long	PK
	placa	String(8)	única, não nula
	marca / modelo	String(60)	não nulos
	ano	int	não nulo
	quilometragem	int	≥ 0
	status	Enum `StatusVeiculo`	DISPONIVEL, ALUGADO, EM_MANUTENCAO
	ativo	boolean	padrão `true`
	categoria_id / filial_id	Long	FK → `categorias` / `filiais`
Opcional (`opcionais`)	id	Long	PK
	nome	String(60)	único, não nulo
	valorDia	BigDecimal(10,2)	≥ 0
Locacao (`locacoes`)	id	Long	PK
	retiradaPrevista / devolucaoPrevista	LocalDateTime	não nulos; devolução > retirada
	retiradaReal / devolucaoReal	LocalDateTime	preenchidos na retirada/devolução
	kmInicial / kmFinal	int	km final ≥ km inicial

Entidade (tabela)	Campo	Tipo	Restrições
	status	Enum `StatusLocacao`	RESERVADA, ATIVA, FINALIZADA, CANCELADA
	valorTotal	BigDecimal(10,2)	calculado
	motivoCancelamento	String(200)	opcional
	cliente_id / veiculo_id	Long	FK, não nulas
	filial_retirada_id / filial_devolucao_id	Long	FK → `filiais`
LocacaoOpcional (`locacao_opcionais`)	id	Long	PK
	quantidade	int	≥ 1
	valorDia	BigDecimal(10,2)	valor cobrado no ato
	locacao_id / opcional_id	Long	FK; par único
Pagamento (`pagamentos`)	id	Long	PK
	valor	BigDecimal(10,2)	> 0
	dataPagamento	LocalDateTime	não nulo
	forma	Enum `FormaPagamento`	PIX, CARTAO, DINHEIRO
	tipo	Enum `TipoPagamento`	SINAL, SALDO, MULTA
	locacao_id	Long	FK, não nula
Manutencao (`manutencoes`)	id	Long	PK
	tipo	String(60)	não nulo
	dataInicio / dataFim	LocalDate	fim ≥ início
	custo	BigDecimal(10,2)	≥ 0
	veiculo_id	Long	FK, não nula

Tabela 6 — Dicionário de dados

3.3 Repositórios Spring Data JPA

Cada entidade possui um repositório que estende `JpaRepository<Entidade, Long>`, herdando as operações de CRUD, paginação e ordenação. Foram acrescentadas consultas derivadas do nome do método e consultas JPQL com `@Query` para as regras de negócio:

Repositório	Entidade	Consultas principais (além do CRUD do JpaRepository)
UsuarioRepository	Usuario	findByEmailIgnoreCase, existsByEmailIgnoreCase
ClienteRepository	Cliente	findByCpf, existsByCpf, findByUsuarioId, findWithHabilitacaoById (`@EntityGraph`), busca paginada por nome
HabilitacaoRepository	Habilitacao	findByNumeroCnh, existsByNumeroCnh
CategoriaRepository	Categoria	findByNomeIgnoreCase, existsByNomeIgnoreCase
FilialRepository	Filial	findByCidadeIgnoreCase
VeiculoRepository	Veiculo	findByPlacaIgnoreCase, findByAtivoTrue (paginado), buscarDisponiveis (`@Query` JPQL — RF07/RN01, filtros por filial, categoria e preço)
OpcionalRepository	Opcional	findByNomeIgnoreCase
LocacaoRepository	Locacao	findByClienteId, findByStatus, findDetalhadaById (`@EntityGraph`), existeConflito (`@Query` — RN01)

Repositório	Entidade	Consultas principais (além do CRUD do JpaRepository)
LocacaoOpcionalRepository	LocacaoOpcional	findByLocacaoId
PagamentoRepository	Pagamento	findByLocacaoIdOrderByDataPagamentoAsc, somarPorLocacao (@Query – RN06)
ManutencaoRepository	Manutencao	findByVeiculoIdOrderByDataInicioDesc, existeSobreposicao (@Query – RN01)

Tabela 7 — Repositórios

A camada de persistência é verificada por testes `@DataJpaTest` (banco H2 em memória) que cobrem o cascade 1:1 Cliente/Habilitação, a gravação da relação N:M com opcionais, o cálculo do valor previsto, a detecção de conflito de agenda, a consulta de veículos disponíveis com manutenção, o fluxo reserva → retirada → devolução com multa e o saldo devedor.

3.4 Matriz de Rastreabilidade

Relaciona cada requisito funcional aos endpoints REST previstos para a Etapa 2 e às entidades envolvidas.

RF	Endpoints REST	Entidades envolvidas
RF01	POST /auth/login	Usuario
RF02	GET /usuarios, GET /usuarios/{id}, POST /usuarios, PUT /usuarios/{id}, PATCH /usuarios/{id}/status	Usuario
RF03	POST /clientes, GET /clientes, GET /clientes/{id}, PUT /clientes/{id}, DELETE /clientes/{id}, PUT /clientes/{id}/habilitacao	Cliente, Habilitacao, Usuario
RF04	GET/POST /categorias, GET/PUT/DELETE /categorias/{id}	Categoria
RF05	GET/POST /filiais, GET/PUT/DELETE /filiais/{id}	Filial
RF06	GET/POST /veiculos, GET/PUT/DELETE /veiculos/{id}	Veiculo, Categoria, Filial
RF07	GET /veiculos/disponiveis?inicio=&fim=&filialId=&categoriaId=	Veiculo, Categoria, Filial, Locacao, Manutencao
RF08	GET/POST /opcionais, GET/PUT/DELETE /opcionais/{id}	Opcional
RF09	POST /locacoes	Locacao, LocacaoOpcional, Opcional, Cliente, Habilitacao, Veiculo, Categoria, Filial
RF10	PATCH /locacoes/{id}/retirada	Locacao, Veiculo
RF11	PATCH /locacoes/{id}/devolucao	Locacao, Veiculo, Filial, Categoria
RF12	PATCH /locacoes/{id}/cancelamento	Locacao, Veiculo
RF13	POST /locacoes/{id}/pagamentos, GET /locacoes/{id}/pagamentos	Pagamento, Locacao
RF14	GET/POST /manutencoes, PUT/DELETE /manutencoes/{id}	Manutencao, Veiculo
RF15	GET /locacoes?clienteId=&veiculoId=&filialId=&status=, GET /locacoes/{id}, GET /veiculos/{id}/historico, GET /clientes/{id}/locacoes	Locacao, Cliente, Veiculo, Manutencao

Tabela 8 — Matriz de rastreabilidade RF × endpoints × entidades

4. Repositório e Execução

Código-fonte: https://github.com/brunow27/projeto_web

Estrutura: `src/main/java/br/edu/catolicasc/locafacil/entity` (11 entidades e 5 enums), `.../repository` (11 repositórios) e `src/test/.../PersistenciaTest.java`.

Execução (requer JDK 17+ e Maven): `mvn spring-boot:run` sobe a aplicação com H2 em memória (console em `/h2-console`); `mvn test` executa os testes de persistência. Para PostgreSQL, usar o perfil `postgres` e as variáveis `DB_URL`, `DB_USER` e `DB_PASSWORD`.